Southern New Hampshire University

CS-360— Assignment 6-3: Proof-of-Concept for Android's Accelerometer Sensors

Prepared on: October 10, 2026

Prepared for: Dr. Alejandro Benavente

Prepared by: Alexander Ahmann

Contents

1 Background **1**

2 Accelerometer Proof-of-Concept Walkthrough **2**

2.1 Testing and Preliminary Results 4

2.2 Acknowledgements 4

3 Figures used in this whitepaper **5**

1 Background

Sensors are a set of input peripherals built into the Android mobile phone which allows for the phone to make measurements, or "electronic observations," regarding the "real world" environment. The `SensorManager` (see "Android API Ref-

erence" (n.d.)) is an abstract class[1] that allows for developers to interface with an Android device's sensors. To demonstrate how sensors work, I shall create a proof-of-concept using Android's accelerometer. Like the name implies, the accelerometer measures acceleration, which is the change in the speed of a physical body.[2] I will build a demonstration Android mobile phone application that takes accelerometer measurements, and outputs them onto a `TextView` label as they change.

2 Accelerometer Proof-of-Concept Walkthrough

To create an Android mobile application that embodies the proof-of-concept of a `SensorManager`, I began by first starting up *Android Studio*, and selecting an *Empty Views Activity* written in the Java Language with a minimum Android SDK/API version of 30. When writing the proof-of-concept, I heavily relied on the logic discussed and demonstrated by a "Geeks4Geeks" (2025) article,[3] which includes code snippets for both an accelerometer and a gyroscope.

I mainly borrowed, rather liberally, from the code snippets for their accelerometer as embodied by an `Accelerometer` Java class and source code file. The

[1] `abstract` in the context of Java, at least. I am not quite sure how the `SensorManager` is implemented in Kotila, though.

[2] A formal treatment: given an acceleration vector $\vec{a}(t)$, velocity vector $\vec{v}$, displacement vector $\vec{r}$, and a time scalar t, the acceleration of an object is

$$\vec{a}(t) = \frac{d\vec{v}}{dt} = \frac{d^2\vec{r}}{dt^2}$$

[3] I was initially going to reference an article published by "Kodeco," but then realised that it was written in Kotlin, so I went with the "Geeks4Geeks" tutorial instead; see: Kodeco (Jul. 20, 2020). *Sensors Tutorial for Android: Getting Started*. Retrieved on Oct. 9, 2026 from: https://www.kodeco.com/10838302-sensors-tutorial-for-android-getting-started?page=2#toc-anchor-006

`Accelerometer` class consists of the methods `setListener`, `register`, and `unregister`, which configure the sensor as something of a "background job" to be run whilst the proof-of-concept mobile application is in operation.

The main logic of my application of the accelerometer is in the constructor of the `Accelerometer` class, which uses the `TYPE_LINEAR_ACCELERATION` sensor, and though the `SensorEventListener` and its child `onSensorChanged` method, continuously listening in on the accelerometer sensors on the target phone, and updating the values of the x, y, and z/s coördinates representing the acceleration of the mobile device on their respective axis on a three-dimensional Cartesian plot.

The `MainActivity` activity pane was created to show the measurements taken by the accelerometer sensor, as embodied by the `Accelerometer` class. This is accomplished by first declaring an instance of the class, and registering it as a background service through calling its `register()` method. Then, a listener that periodically monitors the accelerometer sensors for changes in its respective x, y, and z/s values.

Now, three `TextView` widgets have been put onto the `MainActivity` that represent the acceleration in their three respective dimensions in Cartesian space. Every time a change in acceleration is detected, the `setListener` method, and its child `onTranslation` method, changes the three `TextViews` to the correct acceleration currently being measured by the sensor. This process repeats for as long as the proof-of-concept mobile application is in run.[4]

[4]One could posit that, because the mobile application will take measurements indefinately, that the `unregister` method in the `Accelerometer` class is redundant. This is a fair criticism, but I put it in there for the sake of completeness.

2.1 Testing and Preliminary Results

I ran the application in both Android Studio's emulator, and on my personal phone. Figure 3 shows the accelerometer running under *Android Studio*'s Emulator, with constant changes in the x, y, and z/s axes regarding acceleration. I am not sure why this is the case, the Android Emulator should not have any changes in acceleration unless the phone is "moved virtually" through the extended controls. Though one possibility is that the Android Emulator is set up to take sensory input from the developer's workstation's sensoors.[5] Further work will be needed to work out the root cause of why the accelerometer is giving out readings that are not expected in a vacuum.

2.2 Acknowledgements

StackOverflow user "eugene_prg" for their helpful thread on identifying and interacting with activity widgets with `findViewById`[6] and Lokesh Gupta for their tutorial on converting `float` numbers to `Strings`.[7]

[5] A special case of Newton's second law, which is known as his first law, states that: given an absence of external forces F_{ext} acting on the body, it is the case that the acceleration of a given body is at 0, and that the initial velocity of given object is equal to its final velocity,

$$\neg F_{\text{ext}} \Rightarrow \left[\left(\frac{d\vec{v}}{dt} = 0 \right) \wedge \left(\vec{v}_i(t) = \vec{v}_f(t) \right) \right]$$

From this, one can speculate the possibility that the Android Emulator is taking accelerometer inputs from a "hypersensistive" sensor on the developer's workstation. Of course, this does not rule out the other possibilities that the proof-of-concept implementation is incorrect, nor does it rule out the possibility that the Android Emulator is misconfigured.

[6] "eugene_prg" (Jan. 31, 2021). *Address elements by ID instead of findViewById*. StackOverflow. Retrieved on Oct. 9, 2026 from: https://stackoverflow.com/questions/65983738/address-elements-by-id-instead-of-findviewbyid

[7] Gupta, L. (Oct. 4, 2026). *Convert Float to String in Java with N Decimal Places*. How to do it in Java. Retrieved on Oct. 9, 2024 from: https://howtodoinjava.com/java/string/float-to-string-format-decimal-points/

References

API Reference (n.d.). `SensorManager`. Android Developer. Retrieved on Oct. 10, 2026 from:
https://developer.android.com/reference/android/hardware/SensorManager

CS-360 (n.d.). *Module Six Assignment Guidelines and Rubric*. Southern New Hampshire University.

"Geeks4Geeks" (Jul. 23, 2025). *How to Build an Application to Test Motion Sensors in Android*? Retrieved on Oct. 9, 2026 from:
https://www.geeksforgeeks.org/android/how-to-build-an-application-to-test-motion-sensors-in-android/

3 Figures used in this whitepaper

[See the next page ...]

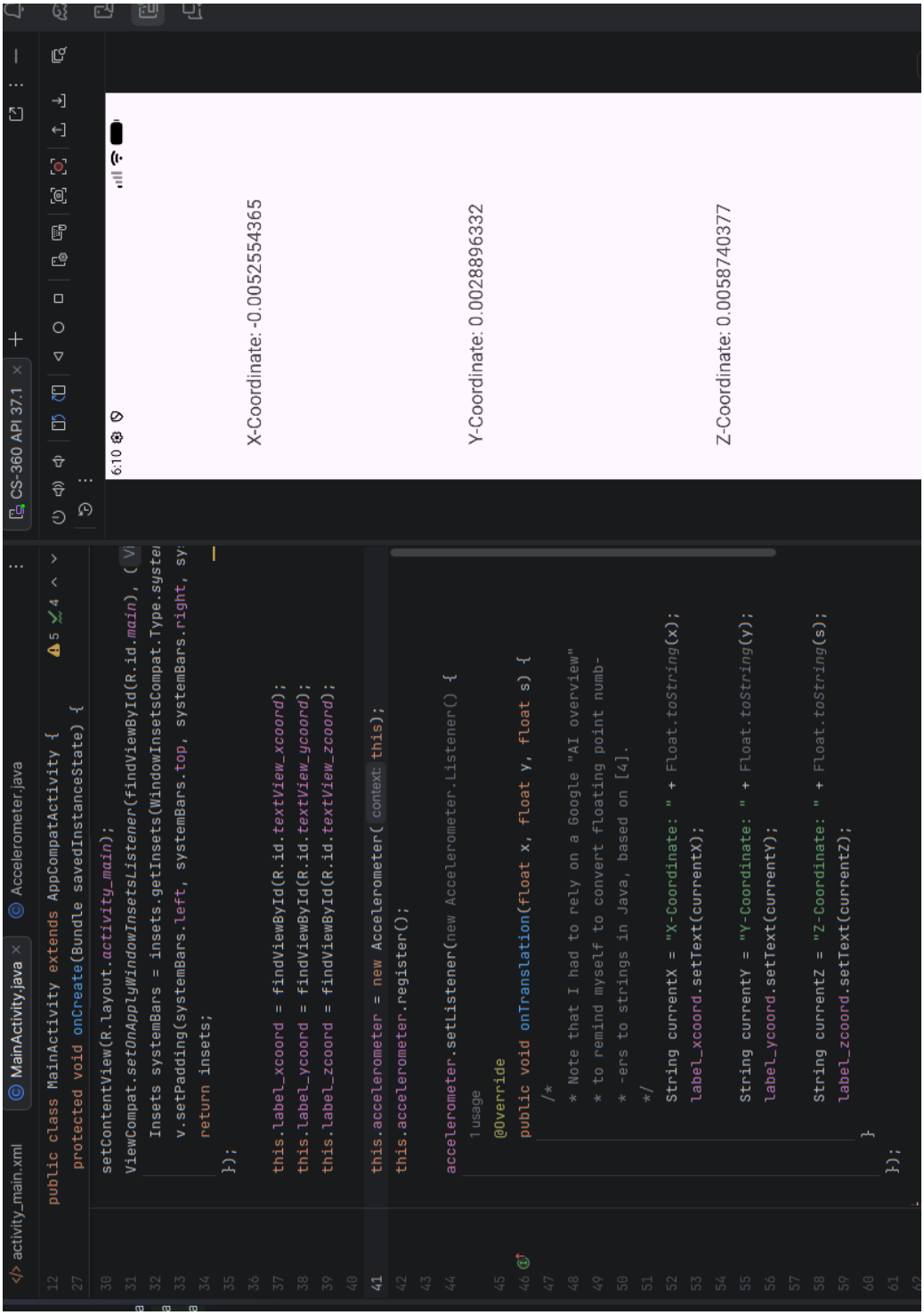


Figure 1: Proof-of-concept Accelerometer running in *Android Studio*'s emulator.